



Instituto de Tecnologia - ITec
Análise e Desenvolvimento de Sistemas

Programação e Otimização de Banco de Dados

**Banco de Dados –
Otimização/Tuning**



Otimização de Banco de Dados

- ☐ A Otimização de um SGBD elimina os possíveis problemas de desempenho existentes em todos os níveis do sistema, isto é:
 - ☐ consultas lentas que eventualmente são submetidas ao banco; configurações do servidor de banco de dados;
 - ☐ Configurações do sistema operacional;
 - ☐ hardware que suportará todo o sistema.

- ☐ Há questões específicas, mas alguns aspectos da otimização se aplicam a todos os SGBDs



ÍNDICES

- ☐ Os índices são estruturas opcionais associadas a tabelas que permitem que as consultas SQL sejam executadas mais rapidamente.
 - ☐ Assim como o índice analítico ou remissivo em um livro
- ☐ Um índice fornece um trajeto de acesso mais rápido aos dados da tabela. A criação dos índices está ligada ao desempenho do banco de dados
- ☐ Índices são sempre bem vindos em colunas de grande seletividade



☐ Criação de índices

- ☐ Podem ser criados automaticamente quando da definição de chaves primárias
- ☐ Declarativamente através de comandos DDL
 - ☐ um índice que seja raramente usado é um desperdício de espaço em disco e também vários índices requerem que para cada atualização num registro seja também feito uma atualização em cada índice.
- ☐ Índices - Sintaxe de criação:
 - ☐ `CREATE INDEX nome_indice ON tabela_nome (coluna_nome);`



ÍNDICES

- ☐ Quando criamos um índice e não especificamos qual o tipo que queremos usar
 - ☐ PostgreSQL utiliza por padrão o tipo **B-Tree**
- ☐ Se desejarmos escolher o tipo de índice, devemos acrescentar ao final o comando USING, logo após especificando o tipo desejado. Exemplo:
 - ☐ CREATE INDEX nome ON tabela USING HASH (coluna);
 - ☐ CREATE INDEX nome ON tabela USING BTREE (coluna);
 - ☐ CREATE INDEX nome ON tabela USING RTREE (coluna);
 - ☐ CREATE INDEX nome ON tabela USING GIST (coluna);

Tipos de índices (descrição):

Índice	Descrição
B-Tree	O índice B-tree é uma implementação das árvores B de alta concorrência. Indicado para consultas com os operadores: <, <=, =, >=, >. Utilizado para indexar colunas que geralmente serão consultadas por intervalo. Por exemplo o campo salário, onde geralmente serão buscado as tuplas por um intervalo de valores desse campo.
R-Tree	Também conhecido como árvores R, utiliza o algoritmo de partição quadrática de Guttman, sendo utilizada para indexar estrutura de dados multidimensionais - e dados espaciais. Utilizado normalmente com dados do tipo box, circle, point e outros. Adequado para consultas com os operadores: <<, <, &>, >>, &&.
HASH	No PostgreSQL testes mostram que os índices possuem desempenho semelhante ou mais lento que os índices B-tree. Os índices hash também possuem um fraco desempenho sob alta concorrência. Utilizado para indexar colunas que serão consultadas por um valor exato.
GIST	Generalized Index Search Trees (Árvores de Procura de Índice Generalizadas). Indicado para hierarquia de conjuntos.



ÍNDICES

- ❑ Na criação de índices podem ser usadas várias colunas (são os índices compostos)
- ❑ Quando se utiliza mais de uma coluna, o índice é organizado de acordo com a primeira coluna especificada na declaração. A 2ª coluna é utilizada quando a primeira coluna possuir vários valores iguais
 - ❑ Assim, a 2ª coluna surge como opção de classificação.
 - ❑ Exemplo:
 - ❑ CREATE INDEX idx_ano_valor ON clientes USING BTREE(ano_nasc, valor_devido);
 - ❑ SELECT nome
 - ❑ FROM clientes
 - ❑ WHERE ano_nasc > 1970 AND valor_devido < 2000;



- ❑ Exemplos:
 - ❑ SELECT nome
 - ❑ FROM clientes
 - ❑ WHERE ano_nasc > 1970 **AND** valor_devido < 2000;
 - ❑ SELECT nome
 - ❑ FROM clientes
 - ❑ WHERE ano_nasc > 1970 **OR** valor_devido < 2000;
- ❑ Por definição, o PostgreSQL utiliza apenas o índice com mais de uma coluna quando estas estão unidas numa cláusula WHERE por **AND**, em outros casos o índice vai ser utilizado apenas na coluna que foi definida por primeiro na criação do índice.
 - ❑ Nos exemplos, apenas a primeiro SELECT usa os dois índices



Comandos com o uso de índices:

- ❑ **Criando um índice**, mas indexando apenas parte do campo
 - ❑ `CREATE INDEX idx_endereco ON pessoas USING btree (substr(endereco, 1, 6));`
- ❑ **Renomeando um índice**:
 - ❑ `CREATE INDEX idx_endereco RENAME TO idx_endereco_completo`
- ❑ **Excluindo (dropando) um índice**:
 - ❑ `DROP INDEX idx_endereco_completo`
- ❑ **Criando um índice com a cláusula WHERE**:
 - ❑ `CREATE UNIQUE INDEX ind_nome ON pessoa USING GIN (nome) WHERE nome ILIKE 'Juca%';`
 - ❑ GIN: tipo de índice três vezes mais rápido (e maior também) que a GiST, indexa a partir das palavras, mas o desempenho cai, recomendável utilizar para indexar nomes
 - ❑ iLIKE: retornará todos os registros que comecem com a letra J seja estando ela em maiúscula ou em minúscula
 - ❑ <http://www.postgresql.org/docs/9.1/static/>

Exemplo



❑ Medindo custos da consulta com **Explain Analyze**

- ❑ o comando `EXPLAIN` para ver o plano de consulta criado pelo sistema para qualquer comando.
- ❑ Os números apresentados pelo Explain são:
 - ❑ O custo de partida estimado (tempo gasto antes de poder começar a varrer a saída).
 - ❑ O custo total estimado (se todas as linhas fossem buscadas, o que pode não acontecer: uma consulta contendo a cláusula `LIMIT`).
 - ❑ Número de linhas de saída estimado (se for executado até o fim).
 - ❑ Largura média estimada (em bytes) das linhas de saída.

❑ Exemplo: `EXPLAIN SELECT * FROM cds`

- ❑ O comando `EXPLAIN` executa uma análise para cada expressão na consulta, retornando uma linha (registro) correspondente a sua análise
- ❑ a variável **cost**, a qual informa algo como o custo de esforço que o PostgreSQL utilizou para atingir o objetivo do comando SQL. Quanto menor o seu valor, mais eficiente a sintaxe foi classificada



ÍNDICES

- ☐ O comando `ANALYZE` coleta estatísticas sobre o conteúdo das tabelas do banco de dados e armazena os resultados na tabela do sistema `pg_statistic`
- ☐ Também é possível verificar a precisão dos custos estimados pelo planejador utilizando o comando `EXPLAIN ANALYZE`
- ☐ Exemplo:
 - ☐ `EXPLAIN ANALYZE SELECT * FROM`
 - ☐ A visualização pode ser feita em milissegundos o tempo de execução que o comando levaria



Dicas de Desempenho:

- ☐ Adicionar índice à tabela (toda chave primária já contém um índice)
- ☐ Adicionar índices aos campos de cláusulas `WHERE`
- ☐ Evitar campos com tamanho variável
- ☐ Evitar índice em tabela muito pequena (poucos registros, não compensa)
- ☐ Evitar, sempre que possível, chaves compostas
- ☐ Separar bancos em um HD e logs em outro HD
- ☐ Aumentar *shared buffers* (`postgresql.conf`) de acordo com RAM disponível (Recomendações: 25% da RAM para *shared buffers* cache).

